Análisis Arquitectónico Formal del Monolito de Referencia y Estrategia de Migración a Arquitecturas Serverless en AWS

Proyecto: Diseño y Evaluación de Agent Skills para Kiro IDE Orientados a la Migración de Aplicaciones Monolíticas a Arquitecturas Serverless en AWS
Autor: Luis Gerardo Sánchez Ordóñez
Director: Juan Gabriel Quintero Barbosa, Mgr. — Head Solutions Architecture, Amazon Web Services Colombia
Institución: Pontificia Universidad Javeriana Cali — Facultad de Ingeniería y Ciencias
Fecha: Agosto de 2026
Tipo de documento: Anexo Técnico — Trabajo de Grado de Maestría en Ingeniería de Software

---

Tabla de Contenido

1. Resumen Ejecutivo
2. Arquitectura del Estado Actual
3. Mapeo de Fronteras de Dominio
4. Análisis de Acoplamiento y Dependencias Compartidas
5. Mapeo Objetivo a Arquitectura Serverless
6. Apéndice: Catálogo Completo de Endpoints

---

1. Resumen Ejecutivo

El presente documento constituye el informe de análisis arquitectónico formal del sistema monolítico de referencia empleado en la investigación de tesis de maestría. Su propósito es doble: servir como evidencia académica del estado inicial del sistema antes de la intervención con *Agent Skills* , y proporcionar el insumo técnico que el Skill 1 (Analizador de Monolito) reproduzca de forma automatizada mediante análisis estático de código fuente.

La aplicación analizada es un sistema de gestión de taller de reparaciones electrónicas (*Taller PyME*), desarrollado en Python con el framework Flask 2.3.3. El sistema está compuesto por cinco módulos funcionales implementados como blueprints de Flask, exponen 29 endpoints REST, gestiona cuatro entidades de dominio persistentes mediante Flask-SQLAlchemy 3.1.1 sobre SQLite (entorno de desarrollo) con capacidad declarada de migración a PostgreSQL, e implementa autenticación mediante tokens JWT (Flask-JWT-Extended 4.5.3) con validación de esquemas a través de Marshmallow 3.20.1.

La evaluación arquitectónica revela que el sistema presenta una madurez de diseño característica de aplicaciones de pequeña y mediana empresa (PyME): alta fidelidad en la implementación de la lógica de negocio central, patrones de conveniencia que generan acoplamiento lateral entre módulos, y anti-patrones de estado compartido que constituyen bloqueadores críticos para la migración a una arquitectura sin servidor (*serverless*). El experimento empírico anterior documentó una cobertura de automatización del 62 % al emplear Kiro IDE sin Skills especializado, con 14 brechas residuales que requieren un esfuerzo estimado de 37 a 40 días-persona para su resolución manual.

Los objetivos de migración son: (1) trasladar los cinco módulos funcionales a funciones AWS Lambda independientes; (2) sustituir el almacenamiento de estado en memoria por servicios gestionados de AWS (DynamoDB para la lista negra de tokens, ElastiCache Redis para el caché); (3) migrar la capa de persistencia a Amazon Aurora Serverless v2 con gestión de conexiones mediante RDS Proxy; (4) reemplazar el almacenamiento de archivos adjuntos en el sistema de archivos local por Amazon S3 mediante el protocolo de URL prefirmada; y (5) sustituir el servicio de notificaciones síncrono por un modelo asíncrono basado en Amazon SES desacoplado mediante Amazon SQS.

La estrategia de migración propuesta sigue el patrón *Strangler Fig* (Fowler, 2024), migrando los módulos en orden creciente de complejidad de acoplamiento, con Amazon API Gateway actuando como capa de intercepción que permite la coexistencia del monolito y las funciones Lambda durante la transición.

---

2. Arquitectura del Estado Actual

2.1 Patrón de Fábrica de Aplicación

La aplicación implementa el patrón *Application Factory* a través de la función `create_app(env_name: str)` definida en `app/__init__.py`. Este patrón es la práctica estándar en Flask para soportar múltiples entornos de ejecución y facilitar las pruebas unitarias mediante la instanciación controlada de la aplicación.

La secuencia de inicialización sigue un orden determinístico relevante para la migración:

1. **Carga de configuración:** `app.config.from_object(config[env_name])` resuelve la clase de configuración apropiada mediante el diccionario `config = {'development': DevelopmentConfig, 'production': ProductionConfig, 'default': DevelopmentConfig}`.
2. **Efectos secundarios de sistema de archivos:** `os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)` crea el directorio de carga en el sistema de archivos local durante el arranque. En el entorno de ejecución de AWS Lambda, el sistema de archivos es efímero y está restringido al directorio `/tmp`. Esta operación es semánticamente incompatible con un modelo de despliegue donde múltiples instancias de la función se ejecutan en paralelo sin sistema de archivos compartido.
3. **Inicialización de extensiones:** Las extensiones compartidas se vinculan a la instancia de la aplicación en el orden `db → migrate → jwt → bcrypt`. Este orden es significativo porque Flask-Migrate depende de que SQLAlchemy (`db`) esté inicializado primero.
4. **Registro de blueprints:** Los cinco blueprints se registran con sus prefijos de URL correspondientes mediante `_registrar_blueprints(app)`.
5. **Comandos CLI:** Se registran los comandos `flask seed` y `flask init-db`. Ambos dependen del contexto de aplicación Flask y de la CLI de Flask-Migrate, mecanismos que no existen en el entorno de ejecución de Lambda. Esta dependencia constituye la brecha GAP-05.
6. **Manejadores de error globales:** Se registran manejadores para los códigos HTTP 404, 400 y 500, retornando respuestas JSON en formato estandarizado `{'error': '...'}`.

2.2 Gestión de Configuración

La jerarquía de configuración está definida en `app/config.py` mediante tres clases: `Config` (base), `DevelopmentConfig(Config)` y `ProductionConfig(Config)`. Los parámetros con impacto directo en la migración son los siguientes:

Parámetro	Valor por defecto	Impacto en migración
`SQLALCHEMY_DATABASE_URI`	`sqlite:///taller.db`	Dialecto SQLite incompatible con Aurora PostgreSQL
`JWT_ACCESS_TOKEN_EXPIRES`	`timedelta(hours=24)`	Debe mantenerse constante durante la coexistencia monolito-Lambda
`JWT_SECRET_KEY`	Fallback hardcodeado en código	Debe migrarse a AWS SSM Parameter Store como SecureString
`UPLOAD_FOLDER`	`'uploads'`	Directorio local; debe reemplazarse por bucket S3
`MAX_CONTENT_LENGTH`	16 MB	API Gateway impone límite de 10 MB; archivos mayores deben usar presigned URL
`ALLOWED_EXTENSIONS`	`{'pdf','png','jpg','jpeg','docx'}`	Validación debe implementarse en política de bucket S3

2.3 Arquitectura de Enrutamiento

El enrutamiento se implementa mediante blueprints de Flask registrados bajo prefijos de URL. La distribución de los 29 endpoints por método HTTP y módulo es la siguiente:

Módulo	GET	POST	PUT	PATCH	DELETE	Total
auth	1	3	1	—	—	5
clientes	3	1	1	—	1	6
servicios	3	1	1	—	1	6

Módulo	GET	POST	PUT	PATCH	DELETE	Total
transacciones	2	2	1	1	1	7
reportes	5	—	—	—	—	5
Total	**14**	**7**	**4**	**1**	**3**	**29**

La distribución por nivel de autenticación revela tres categorías para la configuración de API Gateway: (1) **endpoints públicos** sin JWT — 5 endpoints en auth y servicios; (2) **endpoints JWT estándar** para cualquier usuario autenticado — mayoría de clientes, servicios y transacciones; (3) **endpoints JWT con restricción de rol admin** — todos los endpoints de reportes excepto `servicios-top`, operaciones de eliminación en clientes y servicios, y cancelación de órdenes en proceso.

El monolito implementa un mecanismo de autenticación dual en `auth/routes.py`: además de emitir el token JWT, el endpoint `POST /api/auth/login` persiste el identificador del usuario en la sesión Flask mediante `session['usuario_id'] = usuario.id`. Este mecanismo es incompatible con el modelo stateless de Lambda y debe ser suprimido completamente en la versión serverless.

2.4 Capa de Persistencia

El modelo de datos comprende cinco tablas relacionales:

- `usuarios`: empleados del taller (técnicos y administradores). Relación 1:N con `ordenes_trabajo` vía `tecnico_id`.
- `clientes`: personas que llevan equipos al taller. Relación 1:N con `ordenes_trabajo` con `cascade='all, delete-orphan'`.
- `servicios`: catálogo de tipos de reparación. Relación 1:N con `detalles_orden`.
- `ordenes_trabajo`: entidad transaccional central. La relación con `detalles` utiliza `lazy='joined'`, lo que implica que toda consulta que recupere una instancia de `OrdenTrabajo` ejecutará un JOIN automático con `detalles_orden`. El método `to_dict()` accede a `self.cliente.nombre_completo()` y `self.tecnico.nombre`, desencadenando consultas adicionales cuando los objetos relacionados no están en el caché de identidad de la sesión SQLAlchemy — el clásico problema N+1.
- `detalles_orden`: tabla de unión enriquecida entre `ordenes_trabajo` y `servicios`.

2.5 Anti-Patrones de Estado en Memoria

El sistema presenta tres singletons con estado en proceso que constituyen bloqueadores arquitectónicos de primer orden:

Anti-patrón 1 — Lista negra de tokens JWT: La variable `_tokens_revocados = set()` en `auth/routes.py` almacena los JTIs de tokens invalidados. Al ser una estructura en memoria del proceso Python, se pierde con cada reinicio y es invisible para otras instancias de Lambda. El resultado es un mecanismo de revocación con semántica *fail-open*: un token revocado en una instancia sigue siendo válido en cualquier otra instancia (GAP-03).

Anti-patrón 2 — Caché en proceso: El `CacheService` en `app/services/cache_service.py` implementa un diccionario Python con gestión manual de TTL consumido por cuatro blueprints. En Lambda, múltiples instancias concurrentes operan en procesos completamente independientes; las escrituras de caché de una instancia son invisibles para las demás y la invalidación no se propaga.

Anti-patrón 3 — Historial de notificaciones en memoria: El `NotificacionService` acumula el historial de notificaciones en `self._historial = []`. Si se requiere persistencia del historial, debe implementarse en DynamoDB o CloudWatch Logs.

---

3. Mapeo de Fronteras de Dominio

3.1 Módulo de Autenticación (`app/blueprints/auth/`)

Cohesión: Alta. Todos los endpoints operan sobre la entidad `Usuario` y el mecanismo JWT. Las responsabilidades son homogéneas: gestión de credenciales, emisión y revocación de tokens, recuperación del perfil autenticado.

Aislamiento: Bajo. El módulo expone dos artefactos consumidos externamente: la función `obtener_usuario_actual()` exportada como función pública, y la estructura del payload JWT — `identity=str(usuario.id)` con claims adicionales `{'rol': ..., 'email': ...}` — que es un contrato implícito consumido por todos los demás módulos a través de `get_jwt()['rol']` y `get_jwt_identity()`.

Este contrato debe preservarse con precisión en la versión serverless para garantizar interoperabilidad durante la fase de coexistencia del patrón *Strangler Fig*.

Anti-patrones identificados:

1. Lista negra de tokens JWT en memoria (`_tokens_revocados`) — criticidad alta, bloquea la seguridad (GAP-03).
2. Almacenamiento dual en sesión Flask (`session['usuario_id']`) — incompatible con Lambda.
3. Función auxiliar exportada desde el blueprint (`obtener_usuario_actual`) — genera acoplamiento lateral.

Clasificación de migración: Lambda. Todos los endpoints tienen duración inferior a 5 segundos, no requieren estado persistente entre invocaciones, y siguen un patrón orientado a eventos. La externalización de la lista negra a Amazon DynamoDB con atributo TTL resuelve el anti-patrón 1.

3.2 Módulo de Clientes (`app/blueprints/clientes/`)

Cohesión: Alta. Encapsula el ciclo de vida completo de la entidad `Cliente` : creación, lectura, actualización y desactivación lógica (soft delete).

Aislamiento: Medio. Se identifican dos violaciones de aislamiento. La función `verificar_cliente_existe(cliente_id: int)` es exportada e importada directamente por `transacciones/routes.py` . El endpoint `GET /api/clientes/{id}/historial` importa el modelo `OrdenTrabajo` mediante una importación diferida dentro del controlador — indicio de que el desarrollador reconoció el potencial de importación circular. Arquitectónicamente, este endpoint cruza la frontera del dominio de transacciones.

Anti-patrones identificados:

1. Función exportada usada por otro blueprint (`verificar_cliente_existe`) — acoplamiento lateral.
2. Importación diferida de modelo de otro dominio en el controlador.
3. Asignación directa de campos sin capa de servicio (`setattr(cliente, campo, valor)`).

Clasificación de migración: Lambda. Patrón CRUD estándar con operaciones de corta duración. La función `verificar_cliente_existe` debe migrarse a la Lambda Layer compartida `taller-domain-layer` para ser consumida por `taller-transacciones` .

3.3 Módulo de Servicios (`app/blueprints/servicios/`)

Cohesión: Alta. Gestiona el catálogo de tipos de servicio/reparación de forma autónoma.

Aislamiento: Medio. El módulo exporta la constante `CATEGORIAS = ['electronico', 'electrodomestico', 'computacion', 'celular', 'otro']` a nivel de módulo, importada directamente por `reportes/routes.py` para validar el parámetro de filtro por categoría. Este es acoplamiento por constante compartida que debe resolverse mediante una fuente de verdad única (DynamoDB, SSM Parameter Store, o la base de datos misma).

El módulo mezcla endpoints públicos con endpoints protegidos. En la configuración de API Gateway, los tres endpoints públicos (`GET /` , `GET /categorias` , `GET /{id}`) deben ser explícitamente excluidos del autorizador Lambda.

Anti-patrones identificados:

1. Constante de dominio exportada (`CATEGORIAS`) e importada por otro módulo.
2. Uso del singleton `cache_service` con invalidación por prefijo — incompatible con Lambda multi-instancia.

Clasificación de migración: Lambda. Los tres endpoints de lectura pública son candidatos óptimos para el caché de API Gateway, eliminando la necesidad de ElastiCache para el catálogo.

3.4 Módulo de Transacciones (`app/blueprints/transacciones/`)

Cohesión: Media. Aunque todos los endpoints operan sobre `OrdenTrabajo` , el módulo mezcla responsabilidades heterogéneas: gestión del ciclo de vida de órdenes, manejo de archivos adjuntos, ejecución de una máquina de estados, y despacho de notificaciones.

Aislamiento: Bajo. Se identifican cinco dependencias cross-módulo:

1. Importación de función de otro blueprint: `from app.blueprints.clientes.routes import verificar_cliente_existe`.
2. Uso del singleton `cache_service`.
3. Uso del singleton `notificacion_service` como efecto secundario síncrono.
4. Dependencia del contexto Flask: `current_app.config['UPLOAD_FOLDER']`.
5. Acceso a datos de la tabla `servicios` para validar los detalles de una orden nueva.

Anti-patrones identificados:

1. **Almacenamiento de archivos en sistema de archivos local** (`archivo.save(ruta_destino)`) — **bloqueador crítico GAP-02**. El almacenamiento efímero de Lambda no persiste entre invocaciones ni es accesible entre instancias.
2. **Generación de número de orden por timestamp** (`OT-YYYYMMDD-HHMMSS`) — condición de carrera bajo concurrencia (GAP-14).
3. **Lógica de negocio en el controlador:** cálculo del total de la orden en el handler HTTP.
4. **Efecto secundario síncrono:** despacho de notificaciones dentro de la transacción HTTP, añadiendo latencia.
5. **Máquina de estados en el controlador:** el diccionario de transiciones válidas está embebido en el handler de `PATCH /estado`.

Clasificación de migración: Los endpoints CRUD de órdenes se migran a **Lambda**. El endpoint de carga de archivos (`POST /api/transacciones/{id}/adjunto`) requiere **transformación arquitectónica** (protocolo presigned URL de dos pasos) antes de poder ser migrado.

3.5 Módulo de Reportes (`app/blueprints/reportes/`)

Cohesión: Baja. El módulo agrega datos de todos los demás dominios para producir indicadores de negocio. Esta característica transversal es inherente a un módulo de analítica; sin embargo, implica las dependencias de datos más amplias del sistema.

Aislamiento: Bajo. El módulo importa directamente los cinco modelos SQLAlchemy del sistema: `Usuario`, `Cliente`, `Servicio`, `OrdenTrabajo` y `DetalleOrden`. Adicionalmente, importa la constante `CATEGORIAS` de `servicios/routes.py`.

Anti-patrones identificados:

1. **Uso de `func.strftime()` (función SQL de SQLite):** En `GET /api/reportes/ingresos`, la agrupación por período se implementa con `func.strftime(formato_fecha, OrdenTrabajo.fecha_completado)`. Esta función es exclusiva del dialecto SQLite y no existe en PostgreSQL. La equivalencia en PostgreSQL es `func.to_char()` para formateo o `func.date_trunc()` para agrupación temporal. Este es el anti-patrón GAP-04.
2. **Consultas N+1 en `reporte_tecnicos()`:** El endpoint itera sobre todos los técnicos activos y ejecuta tres consultas SQL independientes por cada técnico, en lugar de una única consulta con `GROUP BY`.
3. **Caché en proceso para resultados de analítica:** La sustitución por ElastiCache Redis es especialmente valiosa en este módulo, dado el alto costo computacional de las consultas de reporte.
4. **Importación de constante de otro blueprint:** `from app.blueprints.servicios.routes import CATEGORIAS`.

Clasificación de migración: Lambda. El módulo es read-only (ningún endpoint modifica datos). La corrección del anti-patrón `func.strftime()` es un prerrequisito obligatorio antes de la migración, así como la resolución del patrón N+1 para evitar el agotamiento del pool de conexiones de RDS Proxy.

---

4. Análisis de Acoplamiento y Dependencias Compartidas

4.1 Grafo de Importaciones Cross-Blueprint

Las dependencias en tiempo de importación entre módulos son las siguientes (la flecha indica la dirección de la dependencia):

`transacciones.routes` ⟶ `clientes.routes` [`verificar_cliente_existe`]
`reportes.routes` ⟶ `servicios.routes` [`CATEGORIAS`]
`clientes.routes` ⟶ `models.orden_trabajo` [`OrdenTrabajo`] (importación diferida)
`reportes.routes` ⟶ `models.*` [`Usuario`, `Cliente`, `Servicio`, `OrdenTrabajo`, `DetalleOrden`]
`transacciones.routes` ⟶ `services.cache_service`

`clientes.routes` ⟶ `services.cache_service` `servicios.routes` ⟶ `services.cache_service` `reportes.routes` ⟶ `services.cache_service` `transacciones.routes` ⟶ `services.notificacion_service`

En el monolito, el sistema de importación de Python resuelve estas dependencias en tiempo de arranque y todos los módulos comparten el mismo espacio de proceso y memoria. En una arquitectura serverless donde cada blueprint se despliega como una función Lambda independiente, estas dependencias cross-boundary se resuelven mediante uno de tres patrones:

- **Lambda Layer compartida:** Para código ejecutable localmente en cada función (modelos, utilidades puras). Es la solución adecuada para `app/utils/helpers.py` y los modelos SQLAlchemy.
- **Invocación cross-función:** Para lógica que reside en otra función Lambda. Introduce latencia de red; debe evitarse en el camino crítico de la solicitud.
- **Evento asíncrono:** Para efectos secundarios como el despacho de notificaciones. La función emisora publica un mensaje en SNS/SQS; la función suscriptora procesa el evento de forma independiente.

4.2 Estrategia de Desacoplamiento de Servicios Compartidos

4.2.1 `CacheService` → Amazon ElastiCache for Redis

La interfaz actual del `CacheService` expone: `get(clave)`, `set(clave, valor, ttl_segundos)`, `delete(clave)` e `invalidar_patron(patron)`. La estrategia de reemplazo es:

- **Componente AWS:** Amazon ElastiCache for Redis (cluster mode disabled para desarrollo; cluster mode enabled para alta disponibilidad en producción).
- **Integración con Lambda:** El cliente `redis-py` se inicializa fuera del handler de Lambda (en el ámbito del módulo) para aprovechar la reutilización del contexto de ejecución entre invocaciones calientes. La operación `invalidar_patron` se implementa mediante el comando `SCAN` con pattern matching de Redis.
- **Lambda Layer:** Una capa `taller-cache-layer` encapsula la inicialización del cliente Redis y provee una interfaz compatible con la del `CacheService` actual, minimizando los cambios en los handlers Lambda.
- **Alternativa para endpoints públicos de solo lectura:** Los tres endpoints públicos de servicios son candidatos para el caché de API Gateway con TTL configurable por ruta, eliminando la necesidad de ElastiCache para el catálogo.

4.2.2 `NotificacionService` → Amazon SES + Amazon SQS

La migración desacopla la lógica de notificación del flujo principal mediante publicación asíncrona:

- La función `taller-transacciones` Lambda publica un mensaje en una cola Amazon SQS estándar con el tipo de notificación y los datos del destinatario.
- Una función Lambda independiente `taller-notifications` suscrita a la cola consume los mensajes y despacha correos mediante Amazon SES.
- **Beneficios:** La latencia del endpoint `PATCH /api/transacciones/{id}/estado` se reduce al eliminar el tiempo de envío del correo del camino crítico. Los fallos transitorios en el servicio de correo no afectan la operación de negocio. Los mensajes no entregados se retienen en la cola para reintentos automáticos.

4.2.3 `app/utils/helpers.py` → Lambda Layer `taller-utils-layer`

Función	Dependencia Flask	Acción requerida
`paginar_query()`	`SQLAlchemy.Query.paginate()`	Reimplementar con `LIMIT` / `OFFSET` SQL nativo
`extension_permitida()`	`current_app.config`	Refactorizar para aceptar el conjunto de extensiones como parámetro explícito
`generar_numero_orden()`	`datetime.utcnow()`	Reemplazar por `uuid.uuid4()` para eliminar colisiones bajo concurrencia
`respuesta_ok()` / `respuesta_error()`	`flask.jsonify()`	Reemplazar con construcción manual de dict (Lambda retorna dicts, no objetos Response de Flask)

4.3 Acoplamiento del Modelo de Datos

La entidad `OrdenTrabajo` presenta el mayor *fan-in* del modelo: es referenciada por tres de los cinco módulos. El método `to_dict(incluir_detalles=False)` accede a los atributos relacionales `self.cliente.nombre_completo()` y `self.tecnico.nombre`, desencadenando consultas adicionales si los objetos no están en el caché de identidad de la sesión SQLAlchemy.

En el entorno Lambda con RDS Proxy, donde el pool de conexiones es un recurso finito compartido entre todas las instancias concurrentes de la función, las consultas N+1 representan un riesgo de agotamiento del pool bajo carga. La solución recomendada es el uso de `joinedload` explícito de SQLAlchemy para consultas que serializarán el objeto completo, y `load_only` para consultas de listado que no requieren los datos relacionados.

4.4 Acoplamiento de Autenticación

La estructura del payload JWT — `identity=str(usuario.id)` con claims `{'rol': ..., 'email': ...}` — es un contrato implícito consumido por todos los módulos vía `get_jwt()['rol']`. En la arquitectura propuesta, este acoplamiento se centraliza en el Autorizador Lambda de API Gateway. El autorizador valida el JWT, extrae los claims y los inyecta en `requestContext.authorizer` de API Gateway. Las funciones Lambda downstream consumen estos valores desde el contexto del evento de API Gateway, eliminando la dependencia de Flask-JWT-Extended en los módulos no relacionados con autenticación.

---

5. Mapeo Objetivo a Arquitectura Serverless

5.1 Arquitectura de Funciones Lambda Propuesta

La traducción de los blueprints a funciones Lambda sigue el principio de un blueprint por función, con la excepción del endpoint de carga de archivos que se descompone en una función independiente.

Función Lambda	Blueprint Origen	Endpoints	Auth	Memoria	Timeout	Complejidad
`taller-auth`	auth	5	Mixta	256 MB	10 s	Media
`taller-clientes`	clientes	6	JWT	256 MB	10 s	Baja
`taller-servicios`	servicios	6	Mixta	128 MB	5 s	Baja
`taller-transacciones`	transacciones (sin adjunto)	6	JWT	512 MB	15 s	Alta
`taller-uploads`	transacciones (adjunto)	1+1*	JWT	256 MB	30 s	Muy Alta
`taller-reportes`	reportes	5	JWT admin	512 MB	30 s	Media
`taller-notifications`	notificacion_service	asíncrono	SQS trigger	128 MB	10 s	Baja

*El endpoint de uploads se descompone en dos: generación de presigned URL (Fase 1) y confirmación post-carga (Fase 2).

Ausencia de Fargate y App Runner: El análisis de los patrones de invocación de los 29 endpoints revela que ningún componente requiere ejecución superior a 15 minutos, estado temporal entre sub-tareas, ni tráfico HTTP continuo con requisitos estrictos de latencia consistente. Todos los componentes satisfacen los criterios de selección de Lambda, haciendo innecesario el uso de Fargate o App Runner para este sistema.

Justificación de memoria: `taller-reportes` y `taller-transacciones` reciben 512 MB por el costo de inicialización de SQLAlchemy con múltiples modelos y por el volumen potencial de datos en operaciones de listado y reporte. `taller-servicios` recibe 128 MB por ser el módulo de menor complejidad computacional.

5.2 Mapeo de Rutas a Amazon API Gateway

Amazon API Gateway REST API actuará como la capa de intercepción del patrón *Strangler Fig*, gestionando enrutamiento, autenticación y autorización.

```
/api ├── /auth │   ├── POST /register          → taller-auth (sin autorizador) │   ├── POST /login
→ taller-auth (sin autorizador) │   ├── GET  /me               → taller-auth (Autorizador Lambda) │
├── POST /logout           → taller-auth (Autorizador Lambda) │   └── PUT  /password          →
taller-auth (Autorizador Lambda) ├── /clientes │   ├── GET  /                → taller-clientes
(Autorizador Lambda) │   ├── POST /                → taller-clientes (Autorizador Lambda) │   ├──
GET  /{id}             → taller-clientes (Autorizador Lambda) │   ├── PUT  /{id}              →
taller-clientes (Autorizador Lambda) │   ├── DELETE /{id}           → taller-clientes (Autorizador
Lambda, claim rol=admin) │   └── GET  /{id}/historial    → taller-clientes (Autorizador Lambda) ├──
/servicios │   ├── GET  /                → taller-servicios (sin autorizador, caché API GW) │   ├──
GET  /categorias       → taller-servicios (sin autorizador, caché API GW) │   ├── GET  /{id}
→ taller-servicios (sin autorizador, caché API GW) │   ├── POST /                → taller-servicios
(Autorizador Lambda) │   ├── PUT  /{id}             → taller-servicios (Autorizador Lambda) │   └──
DELETE /{id}           → taller-servicios (Autorizador Lambda, claim rol=admin) ├── /transacciones │
├── GET  /                → taller-transacciones (Autorizador Lambda) │   ├── POST /
→ taller-transacciones (Autorizador Lambda) │   ├── GET  /{id}             → taller-transacciones
(Autorizador Lambda) │   ├── PUT  /{id}             → taller-transacciones (Autorizador Lambda) │
├── PATCH /{id}/estado      → taller-transacciones (Autorizador Lambda) │   ├── POST /{id}/upload-url
→ taller-uploads (Autorizador Lambda) [NUEVO - Fase 1] │   ├── POST /{id}/confirmar-adj → taller-
uploads (Autorizador Lambda) [NUEVO - Fase 2] │   └── DELETE /{id}           → taller-transacciones
(Autorizador Lambda) └── /reportes     ├── GET /resumen          → taller-reportes (Autorizador
Lambda, claim rol=admin)     ├── GET /ingresos          → taller-reportes (Autorizador Lambda, claim
rol=admin)     ├── GET /tecnicos          → taller-reportes (Autorizador Lambda, claim rol=admin)
├── GET /servicios-top      → taller-reportes (Autorizador Lambda)     └── GET /estado-cache       →
DEPRECADO
```

Autorizador Lambda compartido: Un único Autorizador Lambda de tipo TOKEN valida el JWT para todos los endpoints protegidos. El autorizador extrae el claim `rol` e inyecta el contexto en `requestContext.authorizer`. Las funciones downstream acceden a `event['requestContext']['authorizer']['rol']`. El autorizador consulta DynamoDB para verificar que el JTI no esté en la lista negra, implementando semántica *fail-closed*.

Nota: El endpoint `GET /api/reportes/estado-cache` retorna estadísticas del singleton `CacheService` en memoria. Al migrar el caché a ElastiCache Redis, este endpoint pierde su propósito. Se recomienda deprecarlo y utilizar las métricas nativas de ElastiCache en CloudWatch para el monitoreo del caché.

5.3 Estrategia de Migración de Persistencia

Origen: SQLite 3.x (desarrollo) / PostgreSQL declarado (producción).
Destino: Amazon Aurora Serverless v2 (compatible con PostgreSQL 15).

Infraestructura requerida (IaC con AWS SAM):

- VPC privada con subnets privadas en dos zonas de disponibilidad.
- Cluster Aurora Serverless v2 con ACU mínima de 0.5 y máxima configurable.
- RDS Proxy vinculado al cluster, con autenticación IAM y gestión de pool de conexiones.
- Security Group para el cluster RDS que solo permite tráfico entrante desde el Security Group de las funciones Lambda en el puerto 5432.

Gestión de migraciones de esquema: Flask-Migrate no puede ejecutarse en Lambda. La estrategia propuesta es generar los scripts Alembic de forma standalone durante el proceso de CI/CD e incorporar la ejecución de `alembic upgrade head` como paso previo al despliegue, ejecutada desde un entorno con acceso VPC (AWS CodeBuild dentro de la VPC, o una Lambda de migración invocada mediante un CloudFormation Custom Resource).

Correcciones de compatibilidad ORM: La única incompatibilidad de dialecto confirmada en el análisis estático es el uso de `func.strftime()` en `reportes/routes.py`. La traducción a PostgreSQL es `func.to_char(columna, formato)` para formateo equivalente. Los formatos de SQLite (`'%Y-%m-%d'`, `'%Y-%m'`, `'%Y-%W'`) tienen equivalentes en PostgreSQL (`'YYYY-MM-DD'`, `'YYYY-MM'`, `'IYYY-IW'`).

5.4 Estrategia de Migración de Almacenamiento de Archivos

El endpoint `POST /api/transacciones/{id}/adjunto` utiliza carga directa `multipart/form-data` al servidor. Este patrón es incompatible con Lambda por tres razones: (1) el sistema de archivos de Lambda es efímero; (2) API Gateway impone un límite de 10 MB en el cuerpo de la solicitud; y (3) el archivo no es accesible entre instancias de la función.

El reemplazo introduce un protocolo de dos fases con Amazon S3:

Fase 1 — Generación de URL prefirmada: El cliente invoca `POST /api/transacciones/{id}/upload-url` con el nombre del archivo y el tipo MIME. La función Lambda `taller-uploads` genera una URL prefirmada de S3 con método PUT, una clave de objeto estructurada (`ordenes/{id}/adjuntos/{uuid}-{nombre_seguro}`), y un tiempo de expiración de 15 minutos. La Lambda retorna la URL prefirmada y la clave del objeto.

Fase 2 — Carga directa a S3 y confirmación: El cliente carga el archivo directamente a S3 usando la URL prefirmada, sin pasar por Lambda ni API Gateway. Una vez completada la carga, el cliente invoca `POST /api/transacciones/{id}/confirmar-adjunto` con la clave del objeto. La Lambda verifica que el objeto exista en S3 y actualiza el campo `archivo_adjunto` en Aurora.

Este protocolo constituye un cambio de contrato externo (*breaking change*) que requiere actualización en todos los clientes de la API. La clave del objeto S3 reemplaza semánticamente a la ruta de archivo local en la columna `archivo_adjunto` .

5.5 Estrategia de Lambda Layers

Nombre de Layer	Contenido	Consumido por
`taller-db-layer`	Modelos SQLAlchemy refactorizados, inicialización de conexión RDS Proxy, gestor de sesiones	Todas las funciones Lambda
`taller-auth-layer`	Validación JWT sin Flask-JWT-Extended, lógica del Autorizador Lambda, consulta DynamoDB	API Gateway Authorizer + `taller-auth`
`taller-utils-layer`	Funciones auxiliares sin dependencias Flask, constantes de dominio (reemplaza la constante `CATEGORIAS` exportada), generador UUID	Todas las funciones Lambda

5.6 Secuencia de Migración — Patrón Strangler Fig

Fase	Módulo	Justificación del orden	Prerrequisitos
1	`servicios`	Menor acoplamiento; 3 endpoints públicos permiten validación sin autenticación; sin efectos secundarios en otros módulos	`taller-db-layer` , `taller-utils-layer`
2	`auth`	Establece el Autorizador Lambda compartido requerido por todas las fases siguientes; externaliza lista negra a DynamoDB	DynamoDB tabla de tokens; SSM Parameter Store para JWT secret
3	`clientes`	Acoplamiento medio; `verificar_cliente_existe` migra a `taller-utils-layer` antes de esta fase	Fases 1 y 2 completadas; `taller-auth-layer`
4	`reportes`	Módulo read-only; corrección de `func.strftime()` es el único prerrequisito técnico	Aurora PostgreSQL operativo; corrección SQL de dialecto; ElastiCache Redis
5	`transacciones`	Mayor complejidad; requiere transformación del endpoint de archivos y desacoplamiento de notificaciones	Fases 1–4 completadas; bucket S3 configurado; cola SQS; `taller-notifications` desplegada

---

6. Apéndice: Catálogo Completo de Endpoints

La siguiente tabla consolida los 29 endpoints con su análisis de migración individual. La columna **Anti-Patrones** referencia los identificadores de la Matriz de Brechas del proyecto.

#	Módulo	Método	Ruta	Auth	Rol	Público	Anti-patrones	Destino AWS	Complejidad
1	auth	POST	/api/auth/register	No	—	Sí	—	`taller-auth`	Baja

#	Módulo	Método	Ruta	Auth	Rol	Público	Anti-patrones	Destino AWS	Complejidad
2	auth	POST	/api/auth/login	No	—	Sí	AP-session-flask, AP-jwt-dual	`taller-auth`	Media
3	auth	GET	/api/auth/me	JWT	any	No	—	`taller-auth`	Baja
4	auth	POST	/api/auth/logout	JWT	any	No	AP-token-blacklist (GAP-03)	`taller-auth`	Media
5	auth	PUT	/api/auth/password	JWT	any	No	—	`taller-auth`	Baja
6	clientes	GET	/api/clientes/	JWT	any	No	AP-cache-memory	`taller-clientes`	Baja
7	clientes	POST	/api/clientes/	JWT	any	No	AP-cache-memory	`taller-clientes`	Baja
8	clientes	GET	/api/clientes/{id}	JWT	any	No	AP-cache-memory	`taller-clientes`	Baja
9	clientes	PUT	/api/clientes/{id}	JWT	any	No	AP-cache-memory, AP-setattr	`taller-clientes`	Baja
10	clientes	DELETE	/api/clientes/{id}	JWT	admin	No	AP-cache-memory	`taller-clientes`	Baja
11	clientes	GET	/api/clientes/{id}/historial	JWT	any	No	AP-cross-domain-import	`taller-clientes`	Media
12	servicios	GET	/api/servicios/	No	—	Sí	AP-cache-memory	`taller-servicios` + API GW cache	Baja
13	servicios	GET	/api/servicios/categorias	No	—	Sí	AP-cache-memory, AP-categorias-const	`taller-servicios` + API GW cache	Baja
14	servicios	GET	/api/servicios/{id}	No	—	Sí	AP-cache-memory	`taller-servicios` + API GW cache	Baja
15	servicios	POST	/api/servicios/	JWT	any	No	AP-cache-memory	`taller-servicios`	Baja
16	servicios	PUT	/api/servicios/{id}	JWT	any	No	AP-cache-memory	`taller-servicios`	Baja
17	servicios	DELETE	/api/servicios/{id}	JWT	admin	No	AP-cache-memory	`taller-servicios`	Baja
18	transacciones	GET	/api/transacciones/	JWT	any	No	AP-cache-memory	`taller-transacciones`	Media
19	transacciones	POST	/api/transacciones/	JWT	any	No	AP-cross-blueprint, AP-order-num-ts (GAP-14), AP-biz-logic-ctrl, AP-sync-notif	`taller-transacciones`	Alta
20	transacciones	GET	/api/transacciones/{id}	JWT	any	No	AP-n-plus-1	`taller-transacciones`	Media

#	Módulo	Método	Ruta	Auth	Rol	Público	Anti-patrones	Destino AWS	Complejidad
21	transacciones	PUT	/api/transacciones/{id}	JWT	any	No	—	`taller-transacciones`	Media
22	transacciones	PATCH	/api/transacciones/{id}/estado	JWT	any	No	AP-state-machine-ctrl, AP-sync-notif	`taller-transacciones`	Alta
23	transacciones	POST	/api/transacciones/{id}/adjunto	JWT	any	No	**AP-local-filesystem (GAP-02, bloqueador crítico)**	`taller-uploads` (transformación requerida)	Muy Alta
24	transacciones	DELETE	/api/transacciones/{id}	JWT	admin/any	No	—	`taller-transacciones`	Baja
25	reportes	GET	/api/reportes/resumen	JWT	admin	No	AP-cache-memory	`taller-reportes`	Media
26	reportes	GET	/api/reportes/ingresos	JWT	admin	No	**AP-strftime-sqlite (GAP-04)**, AP-cache-memory	`taller-reportes`	Alta
27	reportes	GET	/api/reportes/tecnicos	JWT	admin	No	AP-n-plus-1-query, AP-cache-memory	`taller-reportes`	Alta
28	reportes	GET	/api/reportes/servicios-top	JWT	any	No	AP-cache-memory, AP-categorias-import	`taller-reportes`	Media
29	reportes	GET	/api/reportes/estado-cache	JWT	admin	No	AP-cache-memory	**DEPRECADO**	N/A

Leyenda de Anti-Patrones

Código	Descripción	Brecha asociada
AP-token-blacklist	Lista negra JWT en memoria del proceso	GAP-03
AP-session-flask	Persistencia de sesión Flask en el login (doble mecanismo auth)	GAP-06
AP-jwt-dual	Emisión simultánea de JWT y cookie de sesión Flask	GAP-06
AP-cache-memory	Uso del singleton `CacheService` en memoria del proceso	—
AP-setattr	Asignación directa de campos sin capa de servicio	—
AP-cross-domain-import	Importación de modelo de otro dominio en el controlador	—
AP-categorias-const	Constante de dominio exportada e importada cross-módulo	—
AP-cross-blueprint	Función de otro blueprint importada directamente	—
AP-order-num-ts	Generación de número de orden por timestamp (condición de carrera)	GAP-14
AP-biz-logic-ctrl	Lógica de negocio (cálculo de total) en el controlador HTTP	—
AP-sync-notif	Notificación síncrona como efecto secundario del controlador	—
AP-state-machine-ctrl	Máquina de estados embebida en el controlador HTTP	—
AP-local-filesystem	Almacenamiento de archivos en el sistema de archivos local del servidor	GAP-02

Código	Descripción	Brecha asociada
AP-n-plus-1	Acceso a relaciones SQLAlchemy sin carga optimizada	—
AP-n-plus-1-query	Múltiples consultas SQL donde un único JOIN sería suficiente	—
AP-strftime-sqlite	Función SQL `strftime()` exclusiva de SQLite, incompatible con PostgreSQL	GAP-04

Fin del Documento de Análisis Arquitectónico Formal

Este documento fue generado como salida del Skill 1 (Analizador de Monolito) sobre el monolito de referencia del proyecto de investigación de maestría. Su contenido refleja el estado del código fuente analizado en agosto de 2026.